



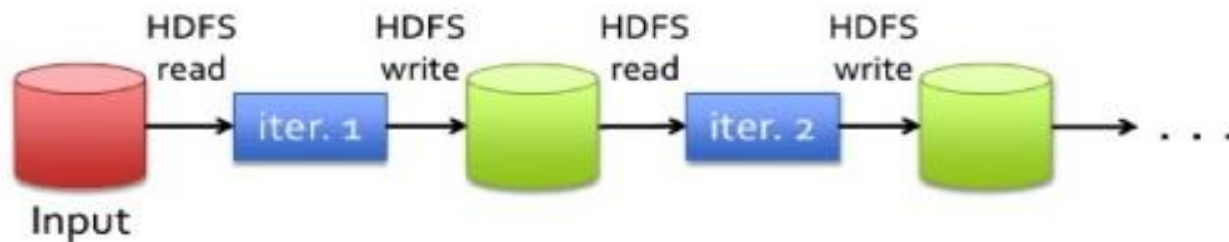
Apache Spark

Một nền tảng xử lý dữ liệu lớn

ONE LOVE. ONE FUTURE.

MapReduce với chuỗi các jobs

- Iterative jobs với MapReduce đòi hỏi thao tác I/O với dữ liệu trên HDFS



- Thực tế I/O trên ổ đĩa cứng rất chậm!

Hard Drive

CrystalDiskMark 3.0.1 x64	
File Edit Theme Help Language	
S 1000MB C: 13% (88/699GB)	
All	Read [MB/s] Write [MB/s]
Seq	112.3 109.3
512K	41.69 48.05
4K	0.543 0.693
4K Qb32	1.004 0.698

SSD

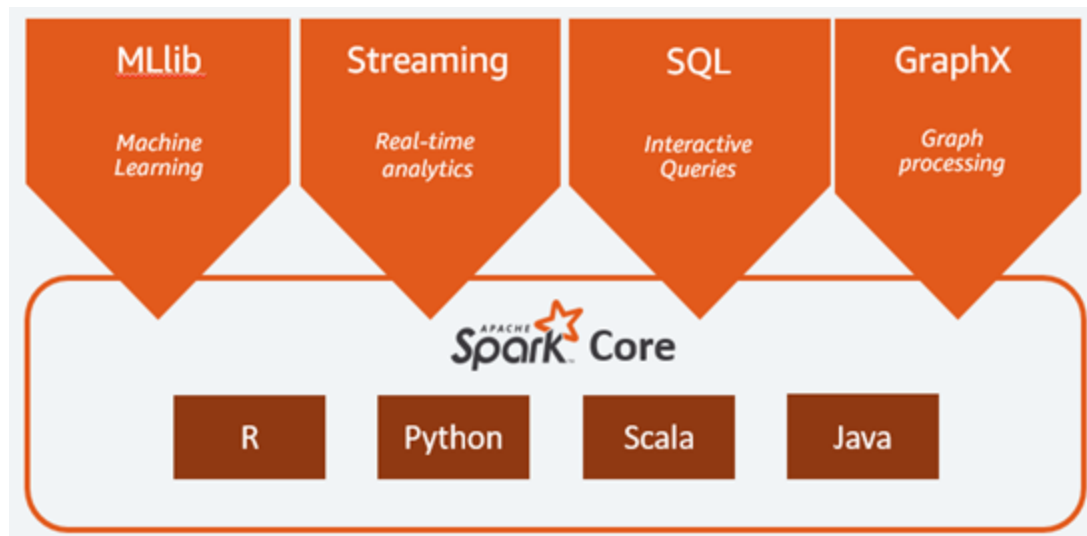
CrystalDiskMark 3.0.1 x64	
File Edit Theme Help Language	
S 1000MB C: 45% (101/224GB)	
All	Read [MB/s] Write [MB/s]
Seq	477.9 235.7
512K	402.7 248.9
4K	30.49 64.67
4K Qb32	200.3 233.2

RAM Disk

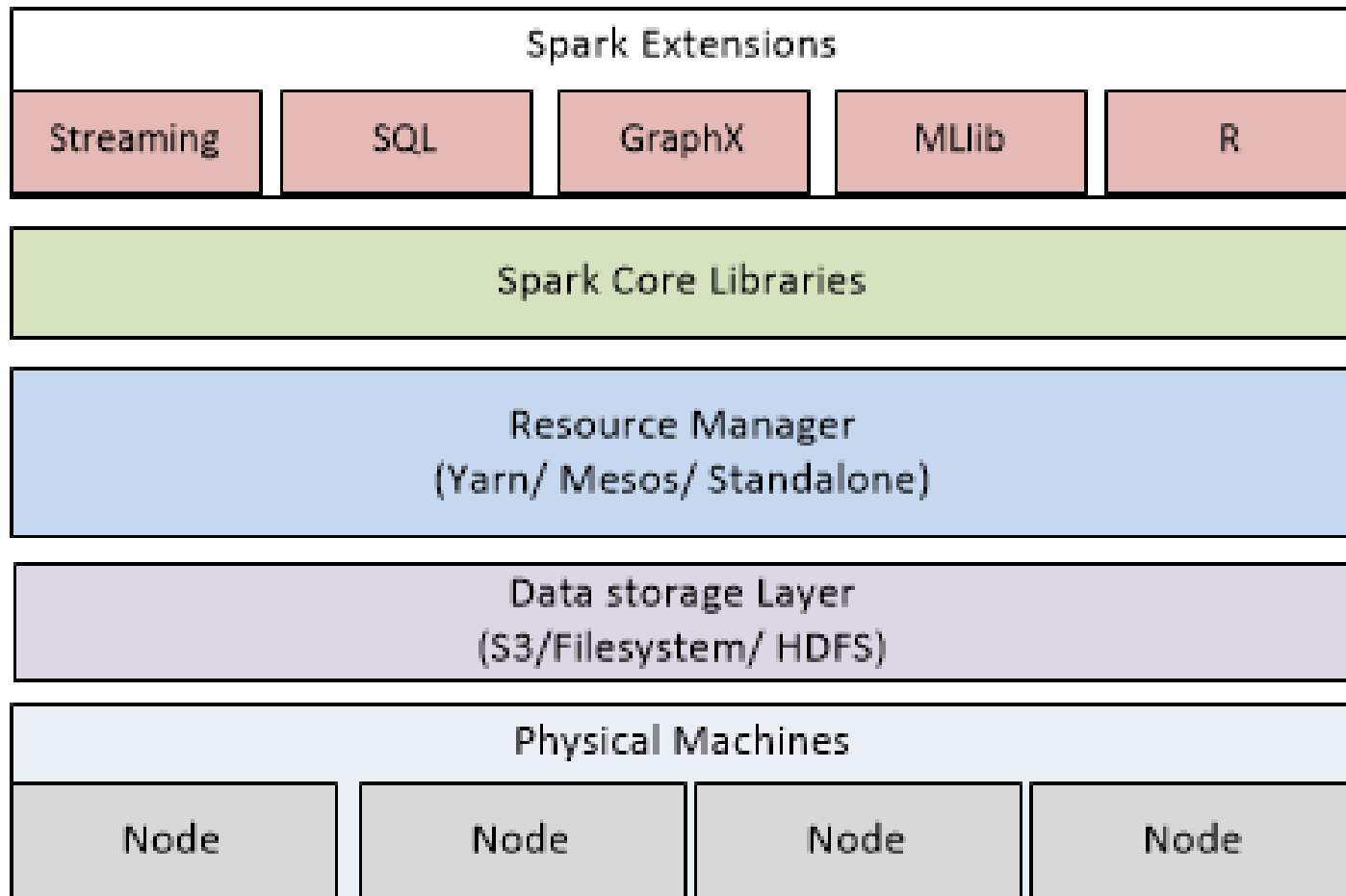
CrystalDiskMark 3.0.1 x64	
File Edit Theme Help Language	
S 1000MB R: 1% (48/4089MB)	
All	Read [MB/s] Write [MB/s]
Seq	5766 7760
512K	5649 7172
4K	657.0 554.8
4K Qb32	631.9 544.7

Một nền tảng xử lý dữ liệu hợp nhất cho dữ liệu lớn

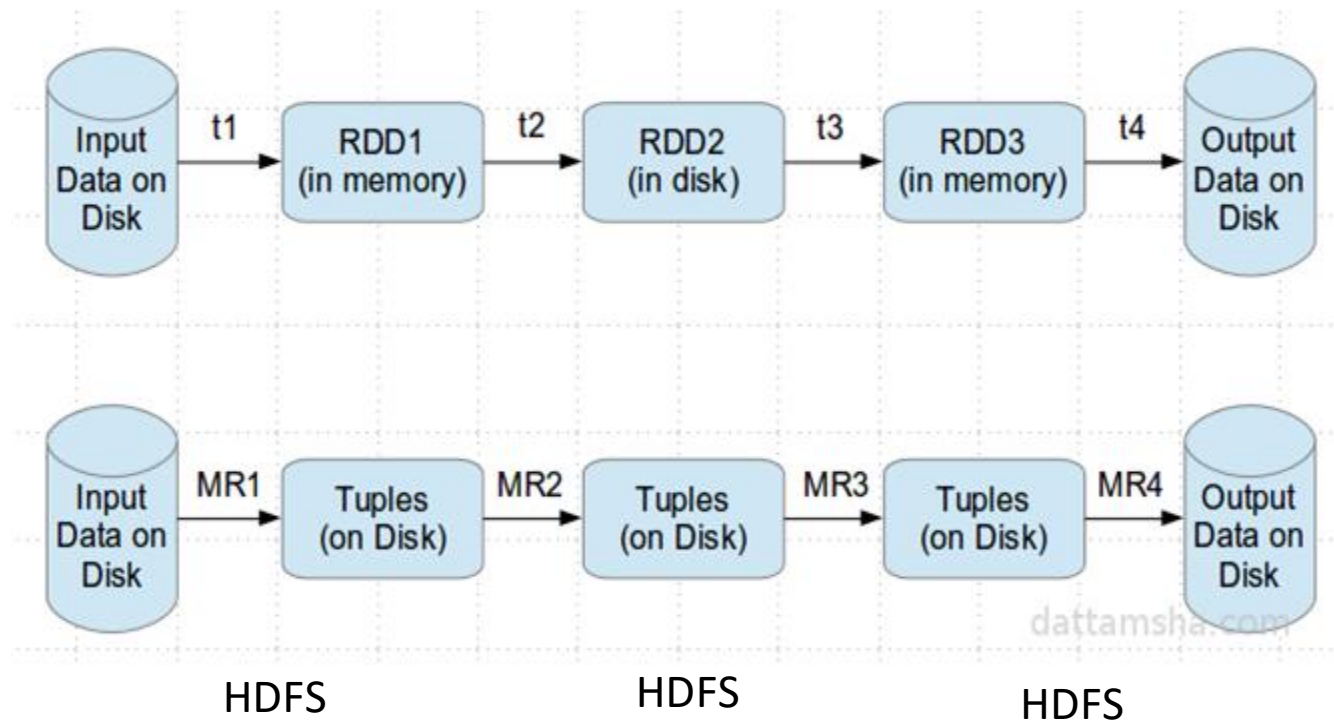
- Hỗ trợ tốt hơn MapReduce trong
 - Các giải thuật có tính lặp - Iterative algorithms
 - Khai phá dữ liệu trong môi trường tương tác
- Ẩn đi sự phức tạp của môi trường phân tán khi lập trình



Architecture



Khai thác bộ nhớ thay vì ổ đĩa HDD



Sự khác nhau giữa Spark và MapReduce

	Apache Hadoop MR	Apache Spark
Storage	Chỉ sử dụng HDD	Sử dụng cả bộ nhớ trong và HDD
Operations	Hai thao tác Map và Reduce	Bao gồm nhiều thao tác biến đổi (transformations) và hành động (actions) trên dữ liệu
Execution model	Xử lý theo khối – batch	Theo khối, tương tác, luồng
Languages	Java	Scala, Java, Python và R

So sánh hiệu năng Spark và MapReduce

	Hadoop World Record	Spark 100 TB *	Spark 1 PB
Data Size	102.5 TB	100 TB	1000 TB
Elapsed Time	72 mins	23 mins	234 mins
# Nodes	2100	206	190
# Cores	50400	6592	6080
# Reducers	10,000	29,000	250,000
Rate	1.42 TB/min	4.27 TB/min	4.27 TB/min
Rate/node	0.67 GB/min	20.7 GB/min	22.5 GB/min
Sort Benchmark Daytona Rules	Yes	Yes	No
Environment	dedicated data center	EC2 (i2.8xlarge)	EC2 (i2.8xlarge)

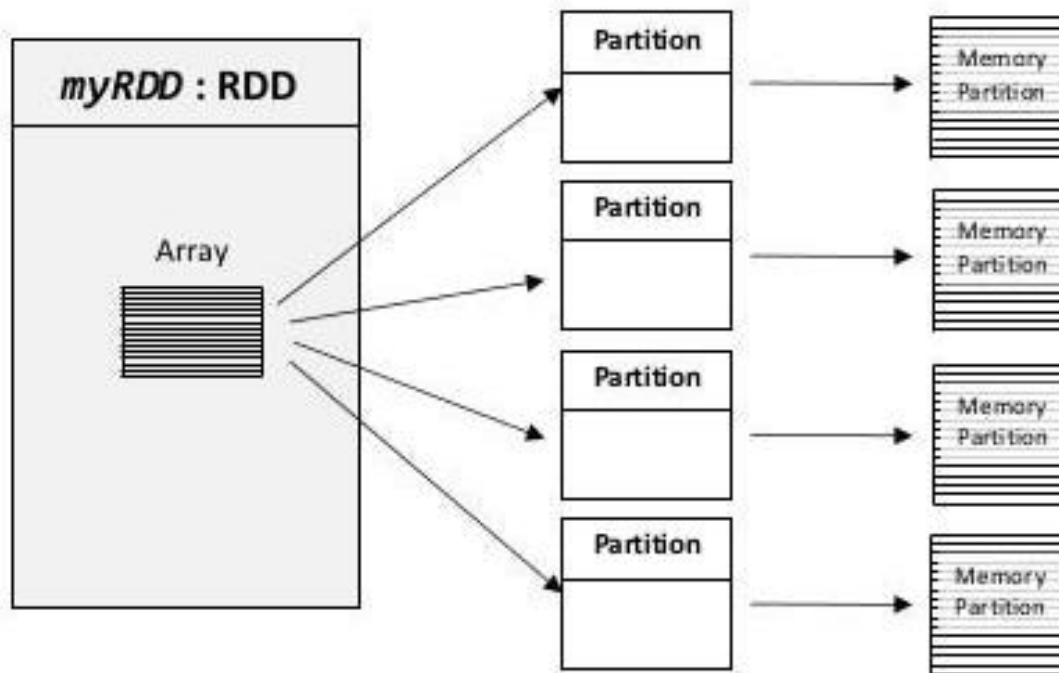
<https://databricks.com/blog/2014/10/10/spark-petabyte-sort.html>



Resilient Distributed Dataset (RDD)

- RDD (tập dữ liệu phân tán có khả năng phục hồi)
- Đặc điểm
 - **Phân tán:** Dữ liệu được chia thành nhiều phân vùng, mỗi phân vùng được xử lý trên một nút riêng biệt.
 - **Bất biến:** RDD không thể thay đổi sau khi được tạo. Mọi thao tác trên RDD sẽ tạo ra một RDD mới.
 - **Chịu lỗi:** RDD tự động khôi phục dữ liệu khi có lỗi nhờ lưu trữ thông tin về cách dữ liệu được tạo ra (lineage).
 - **Tính toán lười (Lazy Evaluation):** Các phép biến đổi (transformations) trên RDD không được thực thi ngay lập tức mà chỉ được ghi nhận, và chỉ thực thi khi có hành động (action) yêu cầu kết quả.

Sự phân vùng của RDD và khả năng song song hóa



Some RDD Characteristics

- Hold references to Partition objects
- Each Partition object references a subset of your data
- Partitions are assigned to nodes on your cluster
- Each partition/split will be in RAM (by default)

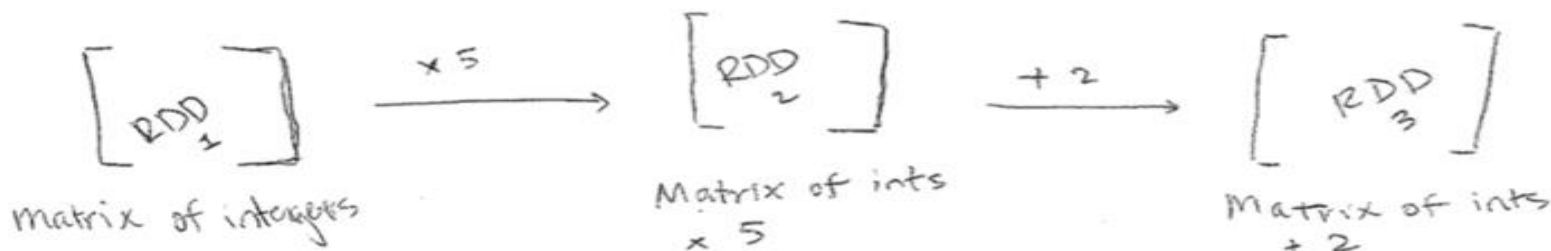
Copyright 2014 Tony Duarte

3

Các thao tác trên RDD

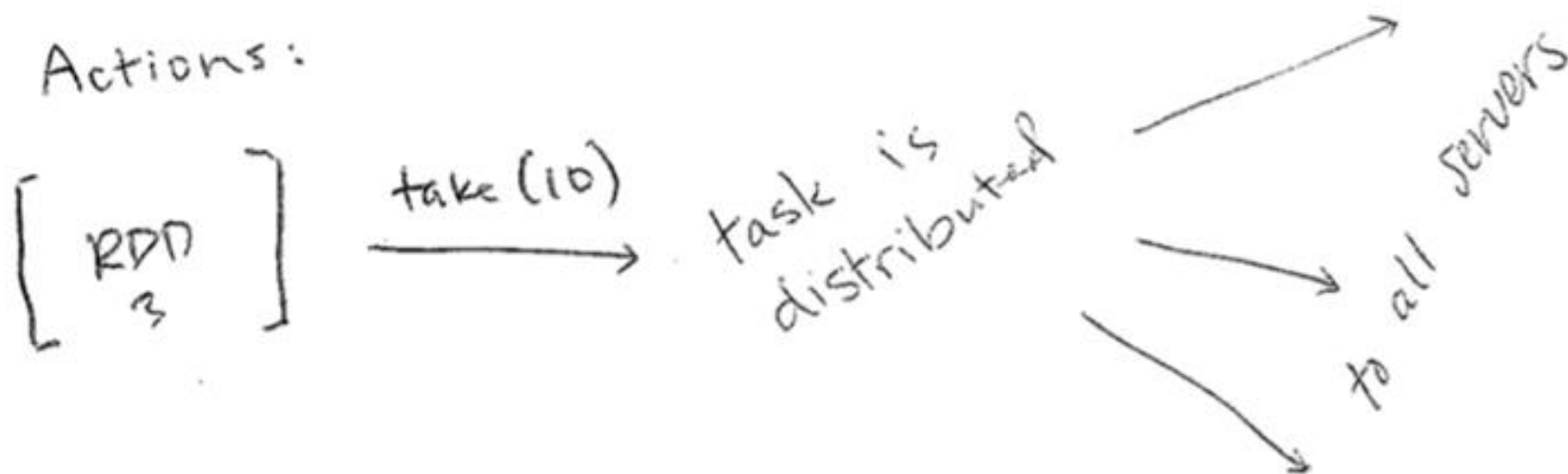
- Hai thao tác: Transformations and Actions
- Transformation
 - Biến đổi một RDD sang một RDD mới
 - Có thể lưu trữ lâu dài (hoặc tạm thời) dữ liệu của RDD trên RAM và Ổ cứng
 - Thao tác: map, filter, distinct,...

Transformations:

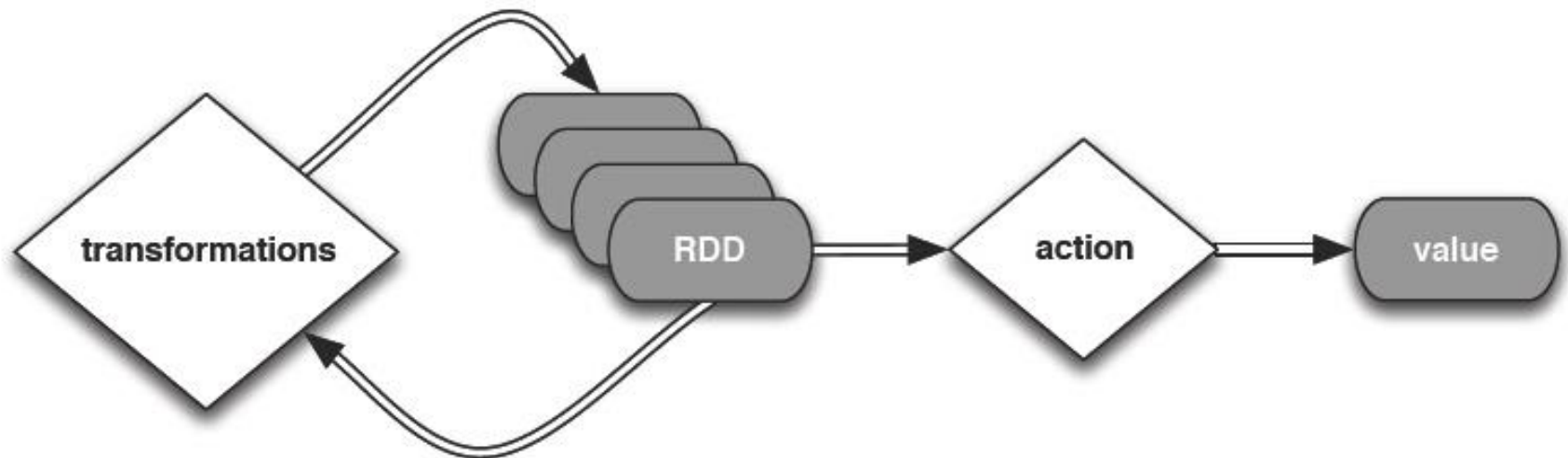


Các thao tác trên RDD

- Hai thao tác: Transformations and Actions
- Actions
 - Tương tác với RDD để có kết quả
 - Thao tác: take, count, saveAsTextFile...



Các thao tác trên RDD



Chịu lỗi sử dụng kỹ thuật Lineage

- Spark lưu mối quan hệ giữa các RDD trong Lineage Graph
 - Sử dụng DAG (Directed Acyclic Graph) để biểu diễn chuỗi phép biến đổi
 - Cho phép khôi phục dữ liệu khi có sự cố
 - Tối ưu hóa thực thi

```
data.txt  
  ↓ (filter)  
rdd2  
  ↓ (map)  
rdd3  
  ↓ (reduceByKey)  
rdd4
```

Một vài ghi nhớ về RDD

- Khởi tạo RDD từ disks (HDFS, etc)
- RDD trung gian nằm trên RAM
- Khôi phục lỗi sử dụng lineage
- Các thao tác trên RDD là phân tán

Quy trình lập trình trên Spark

- Tạo RDD từ nguồn dữ liệu bên ngoài hoặc từ các collections trong bộ nhớ
- Lazily transform thành các RDD trung gian
- cache() RDD để tái sử dụng
- Gọi các actions để thực thi tính toán song song và nhận về các kết quả

Transformation

Table 3-2. Basic RDD transformations on an RDD containing {1, 2, 3, 3}

Function name	Purpose	Example	Result
<code>map()</code>	Apply a function to each element in the RDD and return an RDD of the result.	<code>rdd.map(x => x + 1)</code>	<code>{2, 3, 4, 4}</code>
<code>flatMap()</code>	Apply a function to each element in the RDD and return an RDD of the contents of the iterators returned. Often used to extract words.	<code>rdd.flatMap(x => x.to(3))</code>	<code>{1, 2, 3, 2, 3, 3, 3}</code>
<code>filter()</code>	Return an RDD consisting of only elements that pass the condition passed to <code>filter()</code> .	<code>rdd.filter(x => x != 1)</code>	<code>{2, 3, 3}</code>
<code>distinct()</code>	Remove duplicates.	<code>rdd.distinct()</code>	<code>{1, 2, 3}</code>

Transformation

Table 3-3. Two-RDD transformations on RDDs containing {1, 2, 3} and {3, 4, 5}

Function name	Purpose	Example	Result
<code>union()</code>	Produce an RDD containing elements from both RDDs.	<code>rdd.union(other)</code>	{1, 2, 3, 3, 4, 5}
<code>intersection()</code>	RDD containing only elements found in both RDDs.	<code>rdd.intersection(other)</code>	{3}
<code>subtract()</code>	Remove the contents of one RDD (e.g., remove training data).	<code>rdd.subtract(other)</code>	{1, 2}
<code>cartesian()</code>	Cartesian product with the other RDD.	<code>rdd.cartesian(other)</code>	{(1, 3), (1, 4), ... (3,5)}

Transformation

Table 3-4. Basic actions on an RDD containing {1, 2, 3, 3}

Function name	Purpose	Example	Result
<code>collect()</code>	Return all elements from the RDD.	<code>rdd.collect()</code>	{1, 2, 3, 3}
<code>count()</code>	Number of elements in the RDD.	<code>rdd.count()</code>	4
<code>countByValue()</code>	Number of times each element occurs in the RDD.	<code>rdd.countByValue()</code>	{(1, 1), (2, 1), (3, 2)}

Transformation

Function name	Purpose	Example	Result
<code>take(num)</code>	Return num elements from the RDD.	<code>rdd.take(2)</code>	<code>{1, 2}</code>
<code>top(num)</code>	Return the top num elements the RDD.	<code>rdd.top(2)</code>	<code>{3, 3}</code>
<code>takeOrdered(num)(ordering)</code>	Return num elements based on provided ordering.	<code>rdd.takeOrdered(2)(myOrdering)</code>	<code>{3, 3}</code>
<code>takeSample(withReplacement, num, [seed])</code>	Return num elements at random.	<code>rdd.takeSample(false, 1)</code>	Nondeterministic
<code>reduce(func)</code>	Combine the elements of the	<code>rdd.reduce((x, y) => x + y)</code>	9

Actions

`reduce()`

`collect()`

`count()`

`first()`

`take()`

`takeSample()`

`saveToCassandra()`

`takeOrdered()`

`saveAsTextFile()`

`saveAsSequenceFile()`

`saveAsObjectFile()`

`countByKey()`

`foreach()`

...

Một vài dạng RDD phổ biến

- HadoopRDD
- FilteredRDD
- MappedRDD
- PairRDD
- ShuffledRDD
- UnionRDD
- PythonRDD
- DoubleRDD
- JdbcRDD
- JsonRDD
- VertexRDD
- EdgeRDD
- CassandraRDD
(DataStax)
- GeoRDD
- EsSpark



Spark Streaming

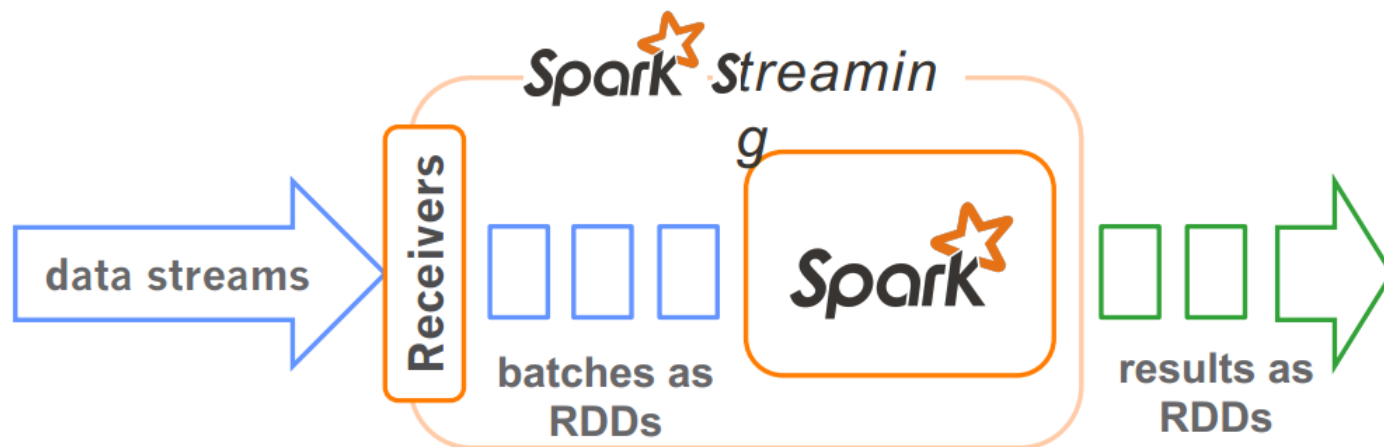
- Spark Streaming là một thành phần của Apache Spark, được thiết kế để xử lý và phân tích dữ liệu thời gian thực/dữ liệu luồng
- Cho phép xử lý dữ liệu liên tục, trong khi tận dụng khả năng xử lý phân tán và song song của Spark



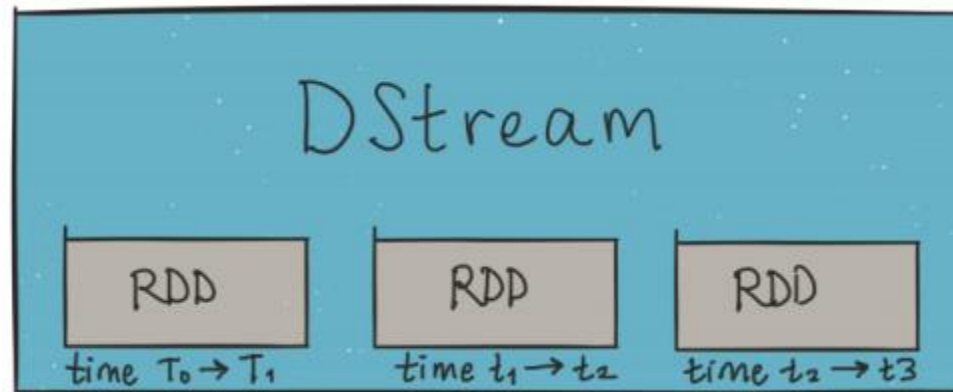
Spark Streaming

Xử lý theo mô hình discretized stream (DStream)

- Dữ liệu luồng được chia tách thành các lô dữ liệu nhỏ
- Mỗi lô được xử lý như RDD, cho phép thao tác như dữ liệu tĩnh

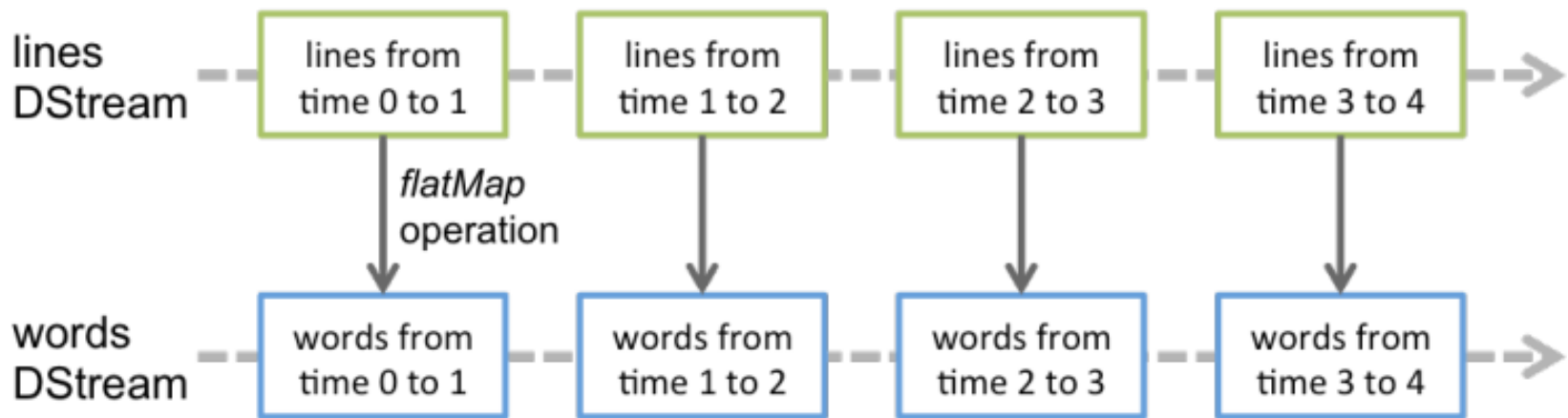


Discretized Stream (DStream)



Discretized Stream (DStream)

Thao tác trên dữ liệu luồng, được chuyển thành thao tác trên RDD



Tổng kết

- Hadoop: Hệ sinh thái lưu trữ và xử lý dữ liệu lớn kinh tế và khả mở
- Spark: Nền tảng phân tích dữ liệu hợp nhất cho dữ liệu lớn